

Laboratorio I

a.a. 2025/2026

Esercitazione su TypeScript I

Esercizio 1: Counter

Si implementi una classe **Counter**, avente un campo pubblico **value** di tipo numerico. La classe **Counter** definisce inoltre un campo statico **default_value**, inizialmente settato a 0. Infine, la classe **Counter** deve inoltre fornire i seguenti metodi:

- Un costruttore che riceva il valore iniziale o **undefined**; se **undefined**, il contatore prende il valore di **default_value**.
- Un metodo **increment(x)** che incrementa **value** di **x**.
- Un metodo **decrement(x)** che decrementa **value** di **x**.

Nei metodi **increment** e **decrement**, se il parametro passato è negativo o uguale a 0, viene lanciata un'eccezione **RangeError**, con opportuno messaggio di errore.

Esercizio 2: Logger

Si definisca una classe **Logger** che offra due metodi statici

- **log(level,message)** per visualizzare un messaggio del tipo “[*level*] message”, dove **level** può essere uno tra “INFO”, “WARNING” e “ERROR”, e **message** è una stringa;
- **history()** per visualizzare nuovamente tutti i messaggi precedentemente visualizzati.

Se il **level** passato è sconosciuto, il metodo **log** lancia un’eccezione **UnknownLevel**, definita opportunamente.

SUGGERIMENTO: Usare una enumerazione per elencare i possibili **level**

Esercizio 3: TreeNode

Implementare una classe **TreeNode** per rappresentare un nodo di albero binario con campi **value** (numero), **left** (altro nodo o **null**) e **right** (altro nodo o **null**). Il costruttore di **TreeNode** prende il numero da assegnare a **value** e inizializza **left** e **right** a **null**. La classe **TreeNode** deve inoltre fornire i seguenti metodi:

- **insertLeft(value)** e **insertRight(value)** per inserire un nuovo nodo con valore **value** a sinistra o destra del nodo corrente, rispettivamente;
- **isLeaf()**, che restituisce **true** se il nodo corrente è una foglia, e **false** altrimenti;
- **minmax()**, che restituisce una tupla contenente i valori minimo e massimo contenuti nell'albero radicato nel **TreeNode** corrente;
- **count()**, che restituisce il numero di nodi presenti nell'albero radicato nel **TreeNode** corrente.

Esercizio 4: Poker

Definire due enum per rappresentare i **Semi** e i **Valori** delle carte da gioco.

Definire poi una classe **Mano** per rappresentare una mano, ovvero un insieme contenente 5 carte da gioco, ciascuna rappresentata come una coppia [**seme, valore**], passate direttamente al costruttore tramite 5 parametri separati. La classe **Mano** deve inoltre fornire i seguenti metodi:

- **poker()**, che restituisce **true** se la mano corrisponde ad un poker, e **false** altrimenti;
- **scala()**, che restituisce **true** se la mano corrisponde ad una scala, e **false** altrimenti;
- **scalaReale()**, che restituisce **true** se la mano corrisponde ad una scala reale, e **false** altrimenti;
- **colore()**, che restituisce **true** se la mano corrisponde ad una colore, e **false** altrimenti;